Python 程式語言

Lecture 10: Scientific Computing Applications

Instructor: 顏家珩 Chia-Heng Yen

Course Roadmap

- Fundamentals & Environment
  - Introduction to Python and Development Environment Setup
  - Basic Syntax and Flow Control
  - Data Structures and Collections
- Advanced Programming
  - Modular and Functional Programming
  - File Handling and Exception Handling
  - Object-Oriented Programming
  - Algorithm Implementation in Python
- User Interface Development
  - GUI Programming with Tkinter and PyQt
- Data Processing & Analysis
  - Data Analysis Fundamentals
  - Scientific Computing Applications
  - Data Visualization
- AI & ML Applications
  - Introduction to Machine Learning with Applications
  - AI-Related Library in Python Applications

Course Objectives

- Master SciPy for scientific computing
- Understand symbolic computation with SymPy
- Implement numerical methods for solving real problems

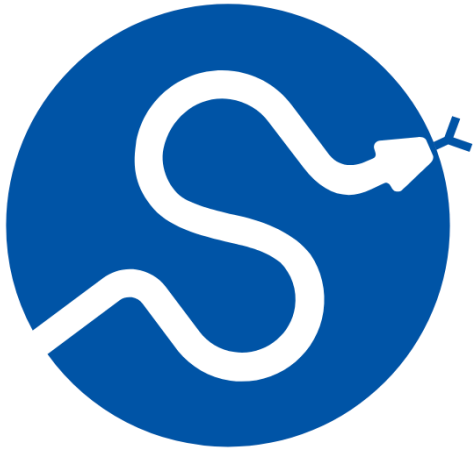
Outline

- SciPy Applications
- SymPy Symbolic Computing
- Numerical Methods Implementation

SciPy Applications

What is SciPy?

- SciPy (Scientific Python) is a library for scientific and technical computing
- Key Features
  - Built on NumPy
  - Provides algorithms for optimization, integration, interpolation, etc.
  - Widely used in engineering and science
  - Efficient and well-tested implementations
- Installation
  - $ pip install scipy
  - $ conda install scipy



SciPy Module Structure

- Main Submodules

```
import scipy
from scipy import linalg          # Linear algebra
from scipy import optimize        # Optimization
from scipy import integrate       # Integration
from scipy import interpolate     # Interpolation
from scipy import signal          # Signal processing
from scipy import stats           # Statistics
from scipy import sparse          # Sparse matrices

print(f"SciPy version: {scipy.__version__}")
```

- Key Concept
  - Each submodule specializes in a specific scientific computing domain

Linear Algebra with SciPy

- Matrix Operations

```
import numpy as np
from scipy import linalg          # Create a matrix
A = np.array([[1, 2], [3, 4]])

# Matrix determinant
det_A = linalg.det(A)
print(f"Determinant: {det_A}")  # -2.0

# Matrix inverse
inv_A = linalg.inv(A)
print(f"Inverse:\n{inv_A}")

# Eigenvalues and eigenvectors
eigenvalues, eigenvectors = linalg.eig(A)
print(f"Eigenvalues: {eigenvalues}")
print(f"Eigenvectors:\n{eigenvectors}")
```

```
Determinant: -2.0
Inverse:
[[-2.   1. ]
 [ 1.5 -0.5]]
Eigenvalues: [-0.37228132+0.j  5.37228132+0.j]
Eigenvectors:
[[-0.82456484 -0.41597356]
 [ 0.56576746 -0.90937671]]
```

Solving Linear Systems

- System of Equations

```
from scipy import linalg

# Solve Ax = b
# Example:
# 2x + 3y = 8
# 3x + 4y = 11

A = np.array([[2, 3], [3, 4]])
b = np.array([8, 11])

# Solve the system
x = linalg.solve(A, b)
print(f"Solution: x = {x[0]:.2f}, y = {x[1]:.2f}")
# Solution: x = 1.00, y = 2.00

# Verify the solution
print(f"Verification: {np.allclose(A @ x, b)}")     # True
```

```
Solution: x = 1.00, y = 2.00
Verification: True
```

*@ is Python's matrix multiplication operator (Python 3.5+)

Matrix Decomposition

- LU Decomposition

```
from scipy import linalg

# Create a matrix
A = np.array([[4, 3], [6, 3]])

# LU decomposition
P, L, U = linalg.lu(A)

print("Original matrix:")
print(A)
print("\nLower triangular L:")
print(L)
print("\nUpper triangular U:")
print(U)
print("\nPermutation matrix P:")
print(P)

# Verify: P @ A = L @ U
print(f"\nVerification: {np.allclose(P @ A, L @ U)}")
```

$$\underbrace{\begin{bmatrix} & 1 & & \\ & & & 1 \\ & & 1 & \\ 1 & & & \end{bmatrix}}_{P} \underbrace{\begin{bmatrix} 2 & 1 & 1 & 0 \\ 4 & 3 & 3 & 1 \\ 8 & 7 & 9 & 5 \\ 6 & 7 & 9 & 8 \end{bmatrix}}_{A} = \underbrace{\begin{bmatrix} 1 & & & \\ \frac{3}{4} & 1 & & \\ \frac{1}{2} & -\frac{2}{7} & 1 & \\ \frac{1}{4} & -\frac{3}{7} & \frac{1}{3} & 1 \end{bmatrix}}_{L} \underbrace{\begin{bmatrix} 8 & 7 & 9 & 5 \\ & \frac{7}{4} & \frac{9}{4} & \frac{17}{4} \\ & & -\frac{6}{7} & -\frac{2}{7} \\ & & & \frac{2}{3} \end{bmatrix}}_{U}.$$

```
Original matrix:
[[4 3]
 [6 3]]

Lower triangular L:
[[1.         0.        ]
 [0.66666667 1.        ]]

Upper triangular U:
[[6. 3.]
 [0. 1.]]

Permutation matrix P:
[[0. 1.]
 [1. 0.]]
```

Optimization Problems

- Function Minimization

```
from scipy import optimize

# Define objective function
# f(x) = (x - 2)^2 + 1
def objective(x):
    return (x - 2)**2 + 1

# Find minimum
result = optimize.minimize_scalar(objective)

print(f"Minimum at x = {result.x:.4f}")       # x = 2.0000
print(f"Minimum value = {result.fun:.4f}")    # f(x) = 1.0000
print(f"Success: {result.success}")           # True

# Alternative method: bounded optimization
result_bounded = optimize.minimize_scalar(
    objective,
    bounds=(0, 5),
    method='bounded'
)
print(f"Bounded minimum at x = {result_bounded.x:.4f}")
```

```
Minimum at x = 2.0000
Minimum value = 1.0000
Success: True
Bounded minimum at x = 2.0000
```

Multivariable Optimization

- Finding Minimum of Multiple Variables

```
from scipy import optimize

# Rosenbrock function (classic optimization test)
# f(x,y) = (1-x)^2 + 100(y-x^2)^2
def rosenbrock(x):
    return (1 - x[0])**2 + 100 * (x[1] - x[0]**2)**2

# Initial guess
x0 = np.array([0, 0])

# Minimize the function
result = optimize.minimize(rosenbrock, x0, method='BFGS')

print(f"Optimal point: x = {result.x[0]:.4f}, y = {result.x[1]:.4f}")
print(f"Minimum value: {result.fun:.6f}")
print(f"Number of iterations: {result.nit}")

# Known minimum is at (1, 1) with f(1,1) = 0
```

```
Optimal point: x = 1.0000, y = 1.0000
Minimum value: 0.000000
Number of iterations: 19
```

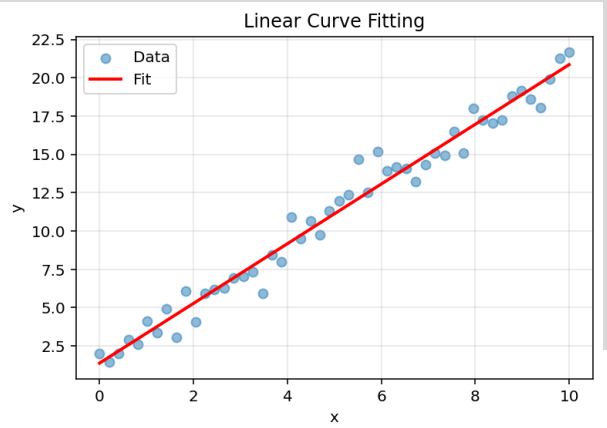
Curve Fitting

- Polynomial Fitting

```
from scipy import optimize
import matplotlib.pyplot as plt
# Generate sample data with noise
x_data = np.linspace(0, 10, 50)
y_true = 2 * x_data + 1
y_data = y_true + np.random.normal(0, 1, 50)
# Add noise

# Define linear model
def linear_model(x, a, b):
    return a * x + b

# Fit the curve
params, covariance =
optimize.curve_fit(linear_model, x_data, y_data)
a_fit, b_fit = params

print(f"Fitted parameters: a = {a_fit:.2f}, b =
{b_fit:.2f}")
print(f"True parameters: a = 2.00, b = 1.00")
```

```
# Plot results
plt.scatter(x_data, y_data, label='Data', alpha=0.5)
plt.plot(x_data, linear_model(x_data, a_fit, b_fit), 'r-',
        label='Fit', linewidth=2)

plt.legend()
plt.xlabel('x')
plt.ylabel('y')
plt.title('Linear Curve Fitting')
plt.grid(True, alpha=0.3)
plt.show()
```

```
Fitted parameters: a = 1.95, b = 1.38
True parameters: a = 2.00, b = 1.00
```



Numerical Integration (數值積分)

- Single Integration

```
from scipy import integrate

# Define function to integrate
# ∫(0 to π) sin(x) dx = 2
def f(x):
    return np.sin(x)
# Compute integral
result, error = integrate.quad(f, 0, np.pi)

print(f"Integral result: {result:.6f}")
print(f"Estimated error: {error:.2e}")
print(f"Expected value: {2.0:.6f}")

# More complex example
# ∫(0 to ∞) e^(-x) dx = 1
def exponential(x):
    return np.exp(-x)

result2, error2 = integrate.quad(exponential, 0, np.inf)
print(f"\nExponential integral: {result2:.6f}")
print(f"Expected: 1.000000")
```

```
Integral result: 2.000000
Estimated error: 2.22e-14
Expected value: 2.000000

Exponential integral: 1.000000
Expected: 1.000000
```

Double Integration

- 2D Integration

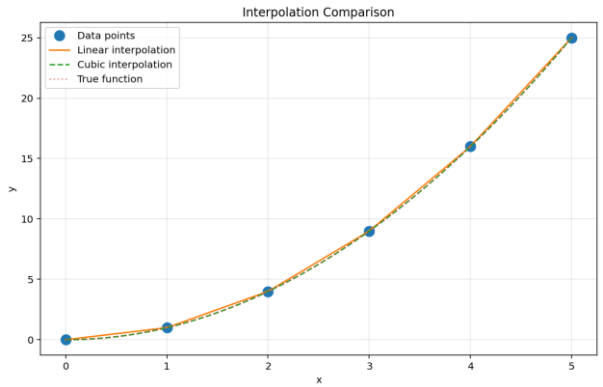
```
from scipy import integrate

# Define 2D function
# ∫∫ x*y dx dy over [0,1] x [0,1]
def integrand(y, x):
    return x * y

# Compute double integral
result, error = integrate.dblquad(
    integrand,
    0, 1, # x bounds
    0, 1 # y bounds
)

print(f"Double integral result: {result:.6f}")
print(f"Estimated error: {error:.2e}")
print(f"Expected value: 0.250000")

# Explanation:
# ∫₀¹ ∫₀¹ xy dx dy = ∫₀¹ [x²y/2]₀¹ dy = ∫₀¹ y/2 dy = [y²/4]₀¹ = 1/4
```

```
Double integral result: 0.250000
Estimated error: 5.54e-15
Expected value: 0.250000
```

Interpolation (插值)

- 1D Interpolation

```
from scipy import interpolate
# Create sample data
x = np.array([0, 1, 2, 3, 4, 5])
y = np.array([0, 1, 4, 9, 16, 25]) # y = x²

# Create interpolation function
f_linear = interpolate.interp1d(x, y, kind='linear')
f_cubic = interpolate.interp1d(x, y, kind='cubic')

# Interpolate at new points
x_new = np.linspace(0, 5, 50)
y_linear = f_linear(x_new)
y_cubic = f_cubic(x_new)
```

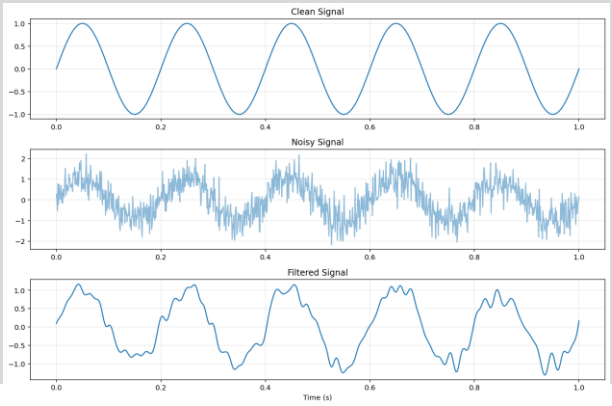



```
# Plot comparison
plt.figure(figsize=(10, 6))
plt.plot(x, y, 'o', label='Data points', markersize=10)
plt.plot(x_new, y_linear, '-', label='Linear interpolation')
plt.plot(x_new, y_cubic, '--', label='Cubic interpolation')
plt.plot(x_new, x_new**2, ':', label='True function', alpha=0.5)
plt.legend()
plt.xlabel('x’)
plt.ylabel('y')
plt.title('Interpolation Comparison')
plt.grid(True, alpha=0.3)
plt.show()
```

Signal Processing

- Filtering Signals

```
from scipy import signal
# Generate noisy signal
t = np.linspace(0, 1, 1000)
clean_signal = np.sin(2 * np.pi * 5 * t)
# 5 Hz sine wave
noise = 0.5 * np.random.randn(len(t))
noisy_signal = clean_signal + noise


# Apply low-pass filter
# Butterworth filter: smooth frequency response
b, a = signal.butter(4, 0.1) # 4th order, cutoff 0.1
filtered_signal = signal.filtfilt(b, a, noisy_signal)
```

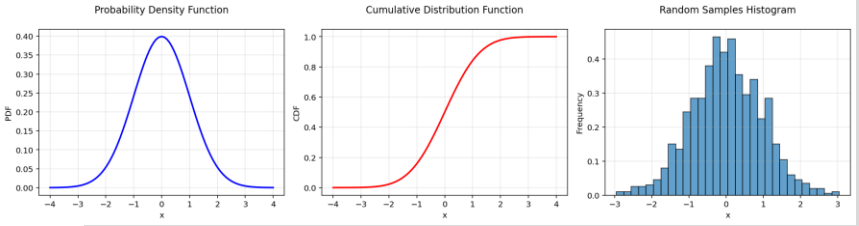



```
# Plot results
fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(12, 8))

ax1.plot(t, clean_signal)
ax1.set_title('Clean Signal')
ax1.grid(True, alpha=0.3)

ax2.plot(t, noisy_signal, alpha=0.5)
ax2.set_title('Noisy Signal')
ax2.grid(True, alpha=0.3)

ax3.plot(t, filtered_signal)
ax3.set_title('Filtered Signal')
ax3.set_xlabel('Time (s)')
ax3.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

Statistical Functions

- Probability Distributions

```
from scipy import stats
# Normal distribution
mu, sigma = 0, 1 # mean and standard deviation
x = np.linspace(-4, 4, 100)

# PDF: Probability Density Function
pdf = stats.norm.pdf(x, mu, sigma)

# CDF: Cumulative Distribution Function
cdf = stats.norm.cdf(x, mu, sigma)

# Generate random samples
samples = stats.norm.rvs(mu, sigma, size=1000)
```




```
# Plot distributions
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(15, 4))
ax1.plot(x, pdf, 'b-', linewidth=2)
ax1.set_title('Probability Density Function\n')
ax1.set_xlabel('x')
ax1.set_ylabel('PDF')
ax1.grid(True, alpha=0.3)
ax2.plot(x, cdf, 'r-', linewidth=2)
ax2.set_title('Cumulative Distribution Function\n')
ax2.set_xlabel('x')
ax2.set_ylabel('CDF')
ax2.grid(True, alpha=0.3)
ax3.hist(samples, bins=30, density=True, alpha=0.7,
edgecolor='black')
ax3.set_title('Random Samples Histogram\n')
ax3.set_xlabel('x')
ax3.set_ylabel('Frequency')
ax3.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

Hypothesis Testing

- T-Test Example

```
from scipy import stats

# Generate two samples
# Sample 1: mean = 5
sample1 = stats.norm.rvs(loc=5, scale=1, size=100)

# Sample 2: mean = 5.3
sample2 = stats.norm.rvs(loc=5.3, scale=1, size=100)

# Perform independent t-test
t_statistic, p_value = stats.ttest_ind(sample1, sample2)

print("T-Test Results:")
print("=" * 50)
print(f"Sample 1 mean: {np.mean(sample1):.4f}")
print(f"Sample 2 mean: {np.mean(sample2):.4f}")
print(f"T-statistic: {t_statistic:.4f}")
print(f"P-value: {p_value:.4f}")
```

```
# Interpretation
alpha = 0.05
if p_value < alpha:
    print(f"\nResult: Reject null hypothesis (p < {alpha})")
else:
    print(f"\nResult: Fail to reject null hypothesis (p >= {alpha})")
```

```
T-Test Results:
==================================================
Sample 1 mean: 5.0888
Sample 2 mean: 5.2154
T-statistic: -0.9709
P-value: 0.3328

Result: Fail to reject null hypothesis (p >= 0.05)
```

SymPy Symbolic Computing

What is Symbolic Computing?

- Symbolic computing manipulates mathematical expressions symbolically
- Numeric vs Symbolic

```
import sympy as sp
# Numeric computation
import math
numeric_result = math.sqrt(8)
print(f"Numeric: √8 = {numeric_result:.6f}") # 2.828427

# Symbolic computation
symbolic_result = sp.sqrt(8)
print(f"Symbolic: √8 = {symbolic_result}") # 2√2

# Simplification
simplified = sp.simplify(symbolic_result)
print(f"Simplified: {simplified}") # 2√2
```

```
Numeric: √8 = 2.828427
Symbolic: √8 = 2*sqrt(2)
Simplified: 2*sqrt(2)
```

- Installation
  - pip install sympy
  - Conda install sympy

Symbolic Variables

- Defining Symbols

```
import sympy as sp

# Define single symbol
x = sp.Symbol('x')
y = sp.Symbol('y')

# Define multiple symbols
a, b, c = sp.symbols('a b c')

# Define symbols with assumptions
n = sp.Symbol('n', integer=True)              # Integer
p = sp.Symbol('p', positive=True)             # Positive
r = sp.Symbol('r', real=True)                 # Real number
print(f"x is integer? {x.is_integer}")        # None
print(f"n is integer? {n.is_integer}")        # True
print(f"p is positive? {p.is_positive}")      # True

# Symbolic expressions
expr = x**2 + 2*x + 1
print(f"Expression: {expr}")
```

```
x is integer? None
n is integer? True
p is positive? True
Expression: x**2 + 2*x + 1
```

Algebraic Operations

- Simplification

```python
import sympy as sp

x, y = sp.symbols('x y')

# Expand expressions
expr1 = (x + y)**2
expanded = sp.expand(expr1)
print(f"Expand: (x+y)² = {expanded}")
# Output: x² + 2xy + y²

# Factor expressions
expr2 = x**2 + 2*x + 1
factored = sp.factor(expr2)
print(f"Factor: x²+2x+1 = {factored}")
# Output: (x+1)²

# Simplify
expr3 = (x**2 - 1)/(x - 1)
simplified = sp.simplify(expr3)
print(f"Simplify: (x²-1)/(x-1) = {simplified}")
# Output: x+1

# Collect terms
expr4 = x*y + x - 3 + 2*x**2 - y*x + 4
collected = sp.collect(expr4, x)
print(f"Collect: {collected}")
```

```
Expand: (x+y)² = x**2 + 2*x*y + y**2
Factor: x²+2x+1 = (x + 1)**2
Simplify: (x²-1)/(x-1) = x + 1
Collect: 2*x**2 + x + 1
```

Solving Equations

- Algebraic Equations

```
import sympy as sp

x = sp.Symbol('x')

# Solve linear equation
# 2x + 3 = 7
solution1 = sp.solve(2*x + 3 - 7, x)
print(f"Linear equation: 2x+3=7")
print(f"Solution: x = {solution1[0]}") # x = 2

# Solve quadratic equation
# x² - 5x + 6 = 0
solution2 = sp.solve(x**2 - 5*x + 6, x)
print(f"\nQuadratic equation: x²-5x+6=0")
print(f"Solutions: x = {solution2}") # [2, 3]
```

```
# Solve system of equations
y = sp.Symbol('y')
# x + y = 5
# 2x - y = 1
solutions = sp.solve([x + y - 5, 2*x - y - 1], [x, y])
print(f"\nSystem of equations:")
print(f"x + y = 5")
print(f"2x - y = 1")
print(f"Solutions: {solutions}") # {x: 2, y: 3}
```

```
Linear equation: 2x+3=7
Solution: x = 2

Quadratic equation: x²-5x+6=0
Solutions: x = [2, 3]

System of equations:
x + y = 5
2x - y = 1
Solutions: {x: 2, y: 3}
```

Calculus: Derivatives (微分)

- Differentiation

```python
import sympy as sp

x = sp.Symbol('x')

# First derivative
f1 = x**3 + 2*x**2 + x
df1_dx = sp.diff(f1, x)
print(f"f(x) = {f1}")
print(f"f'(x) = {df1_dx}") # 3x² + 4x + 1

# Second derivative
d2f1_dx2 = sp.diff(f1, x, 2)
print(f"f''(x) = {d2f1_dx2}") # 6x + 4
```

```python
# Partial derivatives
y = sp.Symbol('y')
f2 = x**2*y + x*y**2
df2_dx = sp.diff(f2, x)
df2_dy = sp.diff(f2, y)
print(f"\nf(x,y) = {f2}")
print(f"∂f/∂x = {df2_dx}") # 2xy + y²
print(f"∂f/∂y = {df2_dy}") # x² + 2xy

# Chain rule example
u = sp.Function('u')
chain = sp.diff(u(x**2), x)
print(f"\nd/dx[u(x²)] = {chain}") # 2x·u'(x²)
```

```
f(x) = x**3 + 2*x**2 + x
f'(x) = 3*x**2 + 4*x + 1
f''(x) = 2*(3*x + 2)

f(x,y) = x**2*y + x*y**2
∂f/∂x = 2*x*y + y**2
∂f/∂y = x**2 + 2*x*y

d/dx[u(x²)] = 2*x*Subs(Derivative(u(_xi_1), _xi_1), _xi_1, x**2)
```

Calculus: Integration (積分)

- Integration Operations

```
import sympy as sp

x = sp.Symbol('x')

# Indefinite integral
f1 = x**2
integral1 = sp.integrate(f1, x)
print(f"∫ x² dx = {integral1}") # x³/3

# Definite integral
# ∫₀¹ x² dx
integral2 = sp.integrate(f1, (x, 0, 1))
print(f"∫₀¹ x² dx = {integral2}") # 1/3
# Multiple integrals
y = sp.Symbol('y')
f2 = x * y
double_integral = sp.integrate(f2, (x, 0, 1), (y, 0, 1))
print(f"∫₀¹ ∫₀¹ xy dx dy = {double_integral}") # 1/4

# Trigonometric integrals
f3 = sp.sin(x)
integral3 = sp.integrate(f3, x)
print(f"∫ sin(x) dx = {integral3}") # -cos(x)

# Improper integrals
f4 = sp.exp(-x)
integral4 = sp.integrate(f4, (x, 0, sp.oo))
print(f"∫₀^∞ e^(-x) dx = {integral4}") # 1
```

```
∫ x² dx = x**3/3
∫₀¹ x² dx = 1/3
∫₀¹ ∫₀¹ xy dx dy = 1/4
∫ sin(x) dx = -cos(x)
∫₀^∞ e^(-x) dx = 1
```

Limits

- Limit Calculations

```python
import sympy as sp
x = sp.Symbol('x')

# Basic limit
# lim(x→0) sin(x)/x
limit1 = sp.limit(sp.sin(x)/x, x, 0)
print(f"lim(x→0) sin(x)/x = {limit1}") # 1

# Limit at infinity
# lim(x→∞) (1 + 1/x)^x
expr = (1 + 1/x)**x
limit2 = sp.limit(expr, x, sp.oo)
print(f"lim(x→∞) (1+1/x)^x = {limit2}") # e

# One-sided limits
# lim(x→0⁺) 1/x
limit3_plus = sp.limit(1/x, x, 0, '+')
print(f"lim(x→0⁺) 1/x = {limit3_plus}") # ∞

# lim(x→0⁻) 1/x
limit3_minus = sp.limit(1/x, x, 0, '-')
print(f"lim(x→0⁻) 1/x = {limit3_minus}") # -∞

# L'Hôpital's rule example
# lim(x→0) (e^x - 1)/x
expr2 = (sp.exp(x) - 1)/x
limit4 = sp.limit(expr2, x, 0)
print(f"lim(x→0) (e^x-1)/x = {limit4}") # 1
```

```
lim(x→0) sin(x)/x = 1
lim(x→∞) (1+1/x)^x = E
lim(x→0⁺) 1/x = oo
lim(x→0⁻) 1/x = -oo
lim(x→0) (e^x-1)/x = 1
```

Series Expansion (級數展開)

- Taylor Series

```
import sympy as sp

x = sp.Symbol('x')
# Taylor series expansion
# Expand around x = 0 (Maclaurin series)
# e^x = 1 + x + x²/2! + x³/3! + ...
exp_series = sp.series(sp.exp(x), x, 0, n=6)
print(f"e^x ≈ {exp_series}")

# sin(x) = x - x³/3! + x⁵/5! - ...
sin_series = sp.series(sp.sin(x), x, 0, n=6)
print(f"sin(x) ≈ {sin_series}")

# cos(x) = 1 - x²/2! + x⁴/4! - ...
cos_series = sp.series(sp.cos(x), x, 0, n=6)
print(f"cos(x) ≈ {cos_series}")

# Expand around different point
# Expand ln(x) around x = 1
ln_series = sp.series(sp.ln(x), x, 1, n=4)
print(f"ln(x) around x=1: {ln_series}")
```

```
e^x ≈ 1 + x + x**2/2 + x**3/6 + x**4/24 + x**5/120 + O(x**6)
sin(x) ≈ x - x**3/6 + x**5/120 + O(x**6)
cos(x) ≈ 1 - x**2/2 + x**4/24 + O(x**6)
ln(x) around x=1: -1 - (x - 1)**2/2 + (x - 1)**3/3 + x + O((x - 1)**4, (x, 1))
```

Matrix Operations

- Symbolic Matrices

```
import sympy as sp
# Define symbolic matrix
a, b, c, d = sp.symbols('a b c d')
M = sp.Matrix([[a, b], [c, d]])

print("Matrix M:")
print(M)

# Determinant
det_M = M.det()
print(f"\nDeterminant: {det_M}") # ad - bc

# Inverse
inv_M = M.inv()
print(f"\nInverse:")
print(inv_M)
# Eigenvalues
eigenvals = M.eigenvals()
print(f"\nEigenvalues: {eigenvals}")
# Eigenvectors
eigenvects = M.eigenvects()
print(f"\nEigenvectors:")
for eigenval, multiplicity, eigenvect in eigenvects:
    print(f"λ = {eigenval}, v = {eigenvect}")
```

```
Matrix M:
Matrix([[a, b], [c, d]])

Determinant: a*d - b*c

Inverse:
Matrix([[d/(a*d - b*c), -b/(a*d - b*c)], [-c/(a*d - b*c), a/(a*d - b*c)]])

Eigenvalues: {a/2 + d/2 - sqrt(a**2 - 2*a*d + 4*b*c + d**2)/2: 1, a/2 + d/2 + sqrt(a**2 - 2*a*d + 4*b*c
+ d**2)/2: 1}

Eigenvectors:
λ = a/2 + d/2 - sqrt(a**2 - 2*a*d + 4*b*c + d**2)/2, v = [Matrix([
[-d/c + (a/2 + d/2 - sqrt(a**2 - 2*a*d + 4*b*c + d**2)/2)/c],
[                                                        1]])]
λ = a/2 + d/2 + sqrt(a**2 - 2*a*d + 4*b*c + d**2)/2, v = [Matrix([
[-d/c + (a/2 + d/2 + sqrt(a**2 - 2*a*d + 4*b*c + d**2)/2)/c],
[                                                        1]])]
```

Numerical Methods Implementation

Root Finding: Bisection Method (求根:二分法)

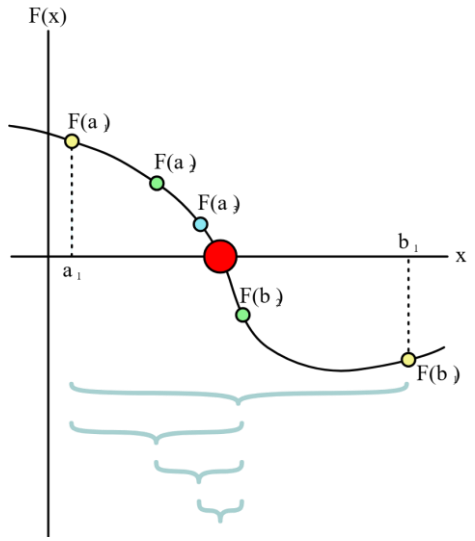
- Bisection Algorithm

```
def bisection(f, a, b, tol=1e-6, max_iter=100):
    """
    Bisection method for finding roots
    Parameters:
        f: function
        a, b: interval [a,b] where f(a)*f(b) < 0
        tol: tolerance
        max_iter: maximum iterations
    Returns:
        Root approximation
    """
    if f(a) * f(b) >= 0:
        raise ValueError("f(a) and f(b) must have opposite signs")

    iteration = 0
    print(f"{'Iter':>4} {'a':>12} {'b':>12} {'c':>12} {'f(c)':>12}")
    print("-" * 56)
    while (b - a) / 2 > tol and iteration < max_iter:
        c = (a + b) / 2 # Midpoint
        fc = f(c)

        print(f"{iteration:4d} {a:12.8f} {b:12.8f} {c:12.8f} {fc:12.8f}")
        if abs(fc) < tol:
            return c
        # Update interval
        if f(a) * fc < 0:
            b = c
        else:
            a = c
        iteration += 1
    return (a + b) / 2
```

Root Finding: Bisection Method

- Bisection Algorithm

```
# Example: Find root of x² - 2 = 0 (√2)
def equation(x):
    return x**2 - 2
print("Finding √2 using Bisection Method")
print("=" * 56)
root = bisection(equation, 1, 2, tol=1e-8)
print(f"\nFound root: {root:.10f}")
print(f"True value: {np.sqrt(2):.10f}")
print(f"Error: {abs(root - np.sqrt(2)):.2e}")
```




```
Finding √2 using Bisection Method
========================================================
Iter           a           b           c        f(c)
--------------------------------------------------------
   0   1.00000000   2.00000000   1.50000000   0.25000000
   1   1.00000000   1.50000000   1.25000000  -0.43750000
   2   1.25000000   1.50000000   1.37500000  -0.10937500
   3   1.37500000   1.50000000   1.43750000   0.06640625
   4   1.37500000   1.43750000   1.40625000  -0.02246094
   5   1.40625000   1.43750000   1.42187500   0.02172852
   6   1.40625000   1.42187500   1.41406250  -0.00042725
   7   1.41406250   1.42187500   1.41796875   0.01063538
   8   1.41406250   1.41796875   1.41601562   0.00510025
   9   1.41406250   1.41601562   1.41503906   0.00233555
  10   1.41406250   1.41503906   1.41455078   0.00095391
  11   1.41406250   1.41455078   1.41430664   0.00026327
  12   1.41406250   1.41430664   1.41418457  -0.00008200
  13   1.41418457   1.41430664   1.41424561   0.00009063
  14   1.41418457   1.41424561   1.41421509   0.00000431
  15   1.41418457   1.41421509   1.41419983  -0.00003884
  16   1.41419983   1.41421509   1.41420746  -0.00001726
  17   1.41420746   1.41421509   1.41421127  -0.00000647
  18   1.41421127   1.41421509   1.41421318  -0.00000108
  19   1.41421318   1.41421509   1.41421413   0.00000162
  20   1.41421318   1.41421413   1.41421366   0.00000027
  21   1.41421318   1.41421366   1.41421342  -0.00000041
  22   1.41421342   1.41421366   1.41421354  -0.00000007
  23   1.41421354   1.41421366   1.41421360   0.00000010
  24   1.41421354   1.41421360   1.41421357   0.00000002
  25   1.41421354   1.41421357   1.41421355  -0.00000003

Found root: 1.4142135605
True value: 1.4142135624
Error: 1.85e-09
```

Newton's Method (牛頓法)

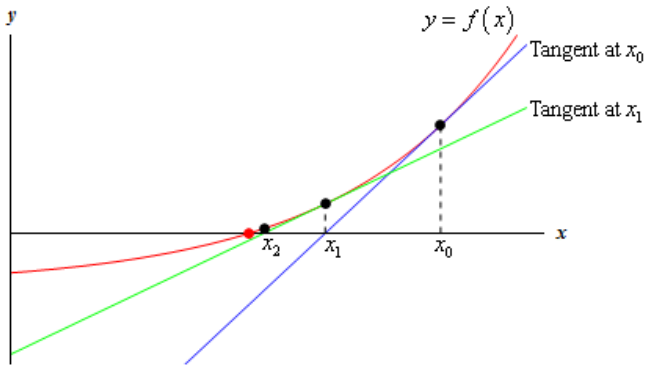
- Newton-Raphson Algorithm

```
def newton_method(f, df, x0, tol=1e-6, max_iter=50):
    """
    Newton's method for finding roots
    Parameters:
        f: function
        df: derivative of f
        x0: initial guess
        tol: tolerance
        max_iter: maximum iterations
    Returns:
        Root approximation
    """
    x = x0
    iteration = 0

    print(f"{'Iter':>4} {'x':>15} {'f(x)':>15} {'f\'(x)':>15}")
    print("-" * 52)

    while iteration < max_iter:
        fx = f(x)
        dfx = df(x)

        print(f"{iteration:4d} {x:15.10f} {fx:15.10f} {dfx:15.10f}")

        if abs(fx) < tol:
            return x
        if abs(dfx) < 1e-12:
            raise ValueError("Derivative too small")
        # Newton's update
        x = x - fx / dfx
        iteration += 1

    return x
```

Newton's Method

- Newton-Raphson Algorithm

```
# Example: Find root of x² - 2 = 0
def f(x):
    return x**2 - 2
def df(x):
    return 2*x

print("\nFinding √2 using Newton's Method")
print("=" * 52)
root_newton = newton_method(f, df, x0=1.5, tol=1e-10)
print(f"\nFound root: {root_newton:.12f}")
print(f"True value: {np.sqrt(2):.12f}")
print(f"Error: {abs(root_newton - np.sqrt(2)):.2e}")
```




```
Finding √2 using Newton's Method
====================================================
Iter               x            f(x)           f'(x)
----------------------------------------------------
   0    1.5000000000    0.2500000000    3.0000000000
   1    1.4166666667    0.0069444444    2.8333333333
   2    1.4142156863    0.0000060073    2.8284313725
   3    1.4142135624    0.0000000000    2.8284271247

Found root: 1.414213562375
True value: 1.414213562373
Error: 1.59e-12
```

Monte Carlo Simulation (蒙地卡羅模擬)

- Estimating π using Monte Carlo

```
def estimate_pi(n_samples):
    """
    Estimate π using Monte Carlo method
    Method:
    - Generate random points in unit square [0,1]×[0,1]
    - Count points inside quarter circle
    - π ≈ 4 × (points inside circle) / (total points)
    """
    # Generate random points
    x = np.random.uniform(0, 1, n_samples)
    y = np.random.uniform(0, 1, n_samples)
    # Check if inside circle
    inside_circle = (x**2 + y**2) <= 1
    # Estimate π
    pi_estimate = 4 * np.sum(inside_circle) / n_samples
    return pi_estimate, x, y, inside_circle

# Run simulation with different sample sizes
sample_sizes = [100, 1000, 10000, 100000]
print("Monte Carlo Estimation of π")
print("=" * 60)
print(f"{'Samples':>10} {'Estimate':>15} {'Error':>15} {'Rel Error %':>15}")
print("-" * 60)
for n in sample_sizes:
    pi_est, _, _, _ = estimate_pi(n)
    error = abs(pi_est - np.pi)
    rel_error = 100 * error / np.pi
    print(f"{n:10d} {pi_est:15.8f} {error:15.8f} {rel_error:15.6f}%")
print(f"\nTrue value of π: {np.pi:.10f}")
```

Monte Carlo Simulation

- Estimating π using Monte Carlo

```
Monte Carlo Estimation of π
============================================================
   Samples        Estimate           Error      Rel Error %
------------------------------------------------------------
       100      3.44000000      0.29840735         9.498601%
      1000      3.11600000      0.02559265         0.814639%
     10000      3.13440000      0.00719265         0.228949%
    100000      3.13636000      0.00523265         0.166561%

True value of π: 3.1415926536
```

```
# Visualize one simulation
n_vis = 5000
pi_est, x, y, inside = estimate_pi(n_vis)

plt.figure(figsize=(8, 8))
plt.scatter(x[inside], y[inside], c='red', s=1, alpha=0.5, label=f'Inside circle ({np.sum(inside)})')
plt.scatter(x[~inside], y[~inside], c='blue', s=1, alpha=0.5, label=f'Outside circle ({np.sum(~inside)})')

# Draw quarter circle
theta = np.linspace(0, np.pi/2, 100)
plt.plot(np.cos(theta), np.sin(theta), 'black', linewidth=2)

plt.xlim(0, 1)
plt.ylim(0, 1)
plt.gca().set_aspect('equal')
plt.xlabel('x')
plt.ylabel('y')
plt.title(f'Monte Carlo Simulation for π\nEstimate: {pi_est:.6f}, True: {np.pi:.6f}')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```